



HORIZON x AEROPOSTALE

Core Web Vitals Audit

WHY WE'RE HERE TODAY

Aeropostale's site has Core Web Vitals (CWV) challenges across all major pages, including home, PLPs, PDPs and cart. Scores and load times indicate meaningful opportunities for search visibility and site CVR. This audit maps the root causes behind current performance trends and **delivers a prioritized roadmap with a clear path toward outpacing key competitors by 20% or more across core benchmarks**



What We'll **Cover**



Our Approach

Page Speed & Lighthouse Insights Over 40 Pages Uncover 86 Issues

PageSpeed Insights

Lab & field data
for all URLs

Google Lighthouse

Detailed audits
per page type

AUDIT SCOPE

Page Types	Homepage · PLP · PDP · Cart · Informational
URLs Tested	5 URLs across mobile (primary focus)
Device Priority	Mobile (70.3% of users, lowest engagement)
Metrics Tracked	LCP · CLS · FCP · TTFB · INP · TBT · Lighthouse Score
CWV Outcome	100% Failure Rate, zero pages pass, 86 Issues
Metrics Tracked	LCP · CLS · FCP · TTFB · INP · TBT · Lighthouse Score

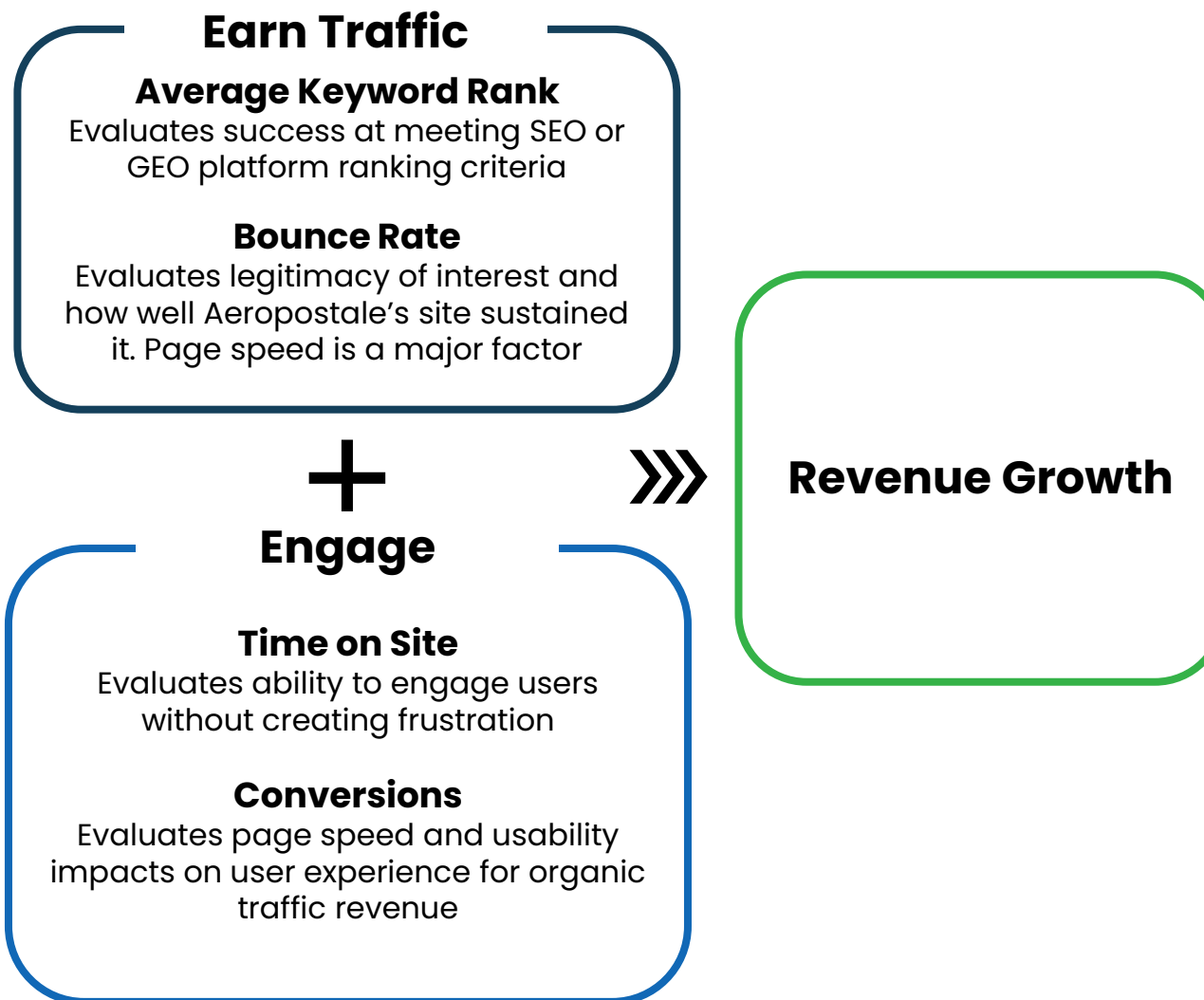
Competitive Benchmark

Target: 20+% faster than all three competitors
across LCP, INP, and CLS

✓ American Eagle

✓ Hollister Co.

Sustained Growth Comes From Improving Multiple KPIs Across The Customer Journey To Drive Revenue



How Performance Metrics Map to Business KPIs

Metric	What It Measures	Organic Traffic	Bounce Rate	Time on Site	Add to Cart	Checkout Conversion	Current Aero Worst-Case
LCP (Largest Contentful Paint)	Speed at which largest above-fold element renders	✓	✓	○	○	○	55s (PDP) 28.7s (PLP)
CLS	Visual stability — how much content shifts during load	✓	✓	✓	✓	✓	0.72 (Homepage) 0.48 (PLP Jeans)
INP	Responsiveness to clicks, taps, and filter interactions	✓	○	✓	✓	✓	High (PLP/Cart) 153s TBT proxy
TBT	Total time main thread is blocked from responding to input	○	✓	✓	✓	✓	153s (PLP Mobile)
TTFB	Total time main thread is blocked from responding to input	✓	✓	○	○	○	990ms (New Arrivals)
FCP	Time until browser renders first visible content on screen	✓	✓	○	○	○	21.5s (PDP) 4.7s (PLP)

Identifying CWV Opportunities By Site Location

KPI	Homepage	PLP (B1G1)	PDP	Cart	Search
Largest Contentful Paint (LCP)	High	High	High	High	High
Cumulative Layout Shift (CLS)	High	Medium	Medium	High	Low
Interaction to Next Paint (INP)	High	High	High	High	High
Total Blocking Time (TBT)	High	High	High	High	High
First Contentful Paint (FCP)	Medium	High	High	High	High
Time to First Byte (TTFB)	High	High	High	High	High

Priority Legend:

High Prio

A high severity issue exists

Medium Prio

A medium severity issue exists

Low Prio

A low severity issue exists

Core Web Vitals Key Opportunities

Three Buckets Drive 100% of CWV Failures



JavaScript Execution & Rendering

All pages load the *entire* toolkit of site code (including shopping cart logic, filters, personalization), even on pages that don't need most of it.



Image Delivery & LCP Prioritization

Hero and product images load with no priority signal and in outdated formats with no content delivery network to speed it up



Third-Party Scripts & Infrastructure

The same tracking package loads on every page — including a size chart and a privacy policy. With no edge caching, every page also starts slower than it should.

Sitewide JavaScript Execution & Rendering Delays Page Load

Scripts Block Home

Hero content is JS-injected rather than built in HTML, delaying largest image load by **18.8 seconds**. With Lighthouse and CLS scores of 5/100 and 0.72, home performance is well below benchmarks.

Product Grids Are Heavy Due To DOM Output

PLPs render 160+ product cards using JavaScript, creating heavy browser work that delays clicks. Total Blocking Time reaches up to 153s, **770x the Google threshold**. All JS loads regardless of page type or scroll position.

Full JS Bundles Load Across All Pages

Informational pages load the same full JS bundle as the PLPs. A simple page fails CWV despite minimal content because all JS loads everywhere, driving a Lighthouse score of 28.

Opportunities

1. Switch Optimizely to async with anti-flicker facade
2. Enable production mode in webpack (NODE_ENV=production)
3. Pre-build the first 4 – 8 PLP product tiles statically in SFCC
4. Split up JS by page type: load Cart JS on the Cart page
5. Defer lazy load on products not visible or near viewport
6. Remove jquery-migrate from production

155s

Peak Script Eval
(Cart)

7.6s

Worst LCP
(Cart Mobile)

2.32s

Homepage LCP
(Mobile)

0.96

Homepage CLS
(Lighthouse)

2

Homepage
Lighthouse Score

148

Scripts loading
sitewide

Large Non-Prioritized Images Load In Poor Formats

LCP Images Are JS-Injected and Not Preloaded

Hero images are injected by JS, not server-rendered. The PDP LCP of 55s is entirely a front-end issue, TTFB was only 340ms, confirming the server responds but the image is not prioritized.

No WebP Format Adoption In PLPs

SFCC DIS is in use but hardcoded to JPEG. No adoption of WebP or AVIF across PLPs, creating image **payloads over 8MB**.

The server is sending 958×1116 Hero images, but displays them at 824×960, so browsers download ~15–20% more data than rendered, compounding LCP delays.

No CDN, Edge Delivery or Responsive Sizing

Images are delivered from origin without srcset or responsive sizing, meaning mobile devices download full-resolution desktop images. TTFB on some PLPs reaches 700–900ms.

Opportunities

1. Add fetchpriority="high" to LCP hero images
2. Add <link rel="preload"> to <head> for above-the-fold images
3. Convert images to WebP/AVIF via SFCC DIS (sfrm=webp)
4. Add srcset with device breakpoints for responsive sizing
5. Remove 1.2 MB desktop nav PNGs from mobile template
6. Preload LCP image in <head> for PDP and Homepage and deploy CDN edge caching for image assets

7.6s

Cart LCP
(Worst on site)

2.51s

PDP/PLP LCP
(Mobile)

69,857 KiB

Homepage Payload
(140× budget)

0%

WebP/AVIF
Adoption confirmed

990ms

Peak TTFB
(New Arrivals PLP)

None

Image CDN / Edge
Caching Not Confirmed

Uncontrolled Script Loading Creates Sitewide Drag

Payment SDKs Load on Every Page (~700 KiB)

Scripts load on all pages. The cart page scored **28 in Lighthouse** as heavy payment, analytics, and 3rd-party scripts that block key interactions load up front, not async

Cart Page at High Business Risk

The cart page fails Core Web Vitals with high TBT driven by payment and cart JS loading on the main thread. Any input delay at checkout directly reduces checkout completion rates.

Opportunities

1. Implement page-type-based script loading (PLP vs PDP)
2. Defer analytics/pixels, payment, stylitics scripts until users might interact (idle callback)
3. Deploy SFCC CDN edge caching (Fastly or Akamai config)
4. Static façade buttons or async-load payment scripts on cart after DOM ready
5. Profile Cart-Show in SFCC Code Profiler
6. Enable basket calculation cache and move tax calls async

~700 KiB

Payment SDKs
on Every Page

4.8s

zenimpact.io
Long Task (Cart)

2.7s

Cart TTFB
(Worst on Site)

19

Cart Lighthouse
Score

210 KiB

Google Maps
on Non-Store Pages

74

Third-Party
Domains

4.7s

Longest critical-path chain
(Stylitics, PDP)

Competitive Benchmark vs Hollister

Metric	Home	PLP	PDP	Cart	Search
LCP (s)	2.32 vs 1.50	2.51 vs 1.00	2.51 vs 1.90	7.60 vs 3.20	3.20 vs 2.00
INP (ms)	387 vs 439	557 vs 623	419 vs 383	295 vs 330	570 vs 438
CLS	0.96 vs 0.17	0.07 vs 0.00	0.07 vs 0.00	0.145 vs 0.18	0.06 vs 0.01
TTFB (ms)	1390 vs 290	1390 vs 280	1480 vs 1110	2690 vs 430	1950 vs 30
LH Score	2 vs 4	26 vs 27	25 vs 25	19 vs 25	26 vs 25

Key Hollister Advantages

- TTFB wins every page: BFF architecture returns sub-300ms on Home/PLP
- Adobe Target (42 KiB) vs Optimizely (123 KiB sync) — 3 orders of magnitude lighter
- No jQuery, no jquery-migrate — modern React stack
- Zero payment SDKs on Home/PLP/Search vs Aero’s ~700 KiB

Where Aero Wins:

- INP on Home, PLP, Cart (slow paint masks interaction delay)
- CLS on Cart (0.145 vs Hollister’s 0.18)
- Forter is 10× lighter CPU than Hollister’s Signifyd
- Both brands fail CWV on all pages — neither passes

Competitive Benchmark vs American Eagle

Metric	Home	PLP	PDP	Cart	Search
LCP (s)	2.32 vs 2.10	2.51 vs 1.30	2.51 vs 2.10	7.60 vs 3.10	3.20 vs 2.10
INP (ms)	387 vs 516	557 vs 471	419 vs 518	295 vs 588	570 vs 518
CLS	0.96 vs 0.14	0.07 vs 0.02	0.07 vs 0.09	0.145 vs 0.88	0.06 vs 0.09
TTFB (ms)	1390 vs 190	1390 vs 80	1480 vs 430	2690 vs 130	1950 vs 1380
LH Score	2 vs 40	26 vs 31	25 vs 35	19 vs 7	26 vs 31

Key American Eagle Advantages

- TTFB wins every page: agwa platform + aggressive edge caching
- Optimizely datafile-only (14 KB JSON) — zero client CPU cost
- Code-split, content-hashed, production webpack — 4–17 KiB minify savings vs Aero’s 837+
- Zero payment SDKs on Home/PLP/Search

Where Aero Wins:

- INP on Home, PDP, Cart (AE’s INP is 471–588ms Poor)
- CLS on PDP, Cart, Search (AE Cart CLS 0.88 = worst in audit)
- Cart Lighthouse score: Aero 19 vs AE 7 (AE’s worst page)
- Both brands fail CWV on every page tested

Implementation Timeline

Quick Wins

- Add fetchpriority="high" to LCP images
- Add <link rel="preload"> for hero images
- Pause / refactor the runaway GTM tag on search results.
- Add explicit width/height to all images
- Reserve layout space for dynamic elements
- Enable production mode in webpack
- Enable brotli compression -- Akamai workflow: properties → content compression behavior → "Last Mile Acceleration" or "HTTP/2"

Metrics: LCP, CLS, TBT

High Impact

- Implement page-type script gating
- Deploy WebP/AVIF image pipeline via SFCC DIS
- Enable CDN edge caching
- Defer analytics until idle/interaction
- No payment script initialization on load: Facade payment buttons rendered statically. Inject SDK for selected method only.
Alt: Async-load cart payment scripts
- Remove jQuery migrate (dev env QA tool) from prod

Metrics: LCP, INP, TTFB

Structural Fixes

- Server-render first 12 PLP product cards (SFCC)
- Server-render cart shell in ISML
- Implement JS code splitting by page type
- Lazy load Stylistics, below-fold product grids, Stylistics, TVPage, Truefit via IntersectionObserver
- Move Optimizely to server-side flags
- Introduce srcset + responsive image sizing

Metrics: TBT, LCP, INP

Scale & Monitor

- Roll out fixes across templates
- Implement real-user monitoring (CrUX dashboard)
- A/B test performance impact on conversion
- Benchmark vs. AE / Hollister bi-annually / annually
- AI SEO: structured data + CWV for AI Overviews

Metrics: All CWV

Quick Hit Implementations to Improve Core Web Vitals

7 low-risk fixes that reduce browser work, prioritize key images, and keep critical shopping paths usable faster

IMAGE DELIVERY & LCP

Prioritize LCP image loading

Add `fetchpriority="high"` to the hero or first product image so the browser finds the most important asset earlier.

KPI: LCP

IMAGE DELIVERY & LCP

Preload above-the-fold hero images

Add a preload tag for the primary image in the page head, including the first product image on PLPs.

KPIs: LCP/FCP

IMAGE DELIVERY & LCP

Reserve image space upfront

Add explicit width and height attributes to product, hero, and badge images to reduce layout shifts as assets load.

KPI: CLS

SCRIPTS & INFRA

Enable Brotli compression at Akamai

Compress JS, CSS, and HTML responses at the CDN or origin layer to reduce transfer size for large bundles.

KPIs: FCP/TBT

JAVASCRIPT EXECUTION

Remove jquery-migrate from prod

Remove tool for Stage QA from prod to prevent 0.15s additional render blocking time on all pages

KPIs: TBT/INP

JAVASCRIPT EXECUTION

Gate scripts by page type

Load ecommerce bundles only where needed so lighter templates do not inherit the full JS stack.

KPIs: TBT/INP

JAVASCRIPT EXECUTION

Enable webpack production mode

(`NODE_ENV=production`). Remove dev-only code, to saves 774–843 KiB on app.js and cut load time across all pages without impacting user experience.

KPIs: TBT/LCP

Recommended sequencing: start with image priority and compression, then move into script controls once page-type rules are confirmed.

Core Web Vitals improvements expand eligibility for AI Overview inclusion and LLM citations



Google AI Overviews Factor Page Quality

Google's AI Overview eligibility criteria increasingly factor in page experience signals



Fast PLPs Rank in AI-Generated Answers

Well-structured, fast-loading PLPs with proper schema markup are more likely to surface when users ask AI assistants shopping questions



CWV Fixes Scale Across PLP Templates

fixing JS rendering and image delivery on the PLP base template unlocks CWV improvements across all 30+ PLP URLs

Recommended Next Steps

Horizon

1 Schedule Sprint 1 dev kickoff

Share this report with the client dev team. Align on Sprint 1 quick wins — fetchpriority, preload, Optimizely defer — and assign dev tickets.

2 Confirm CDN & image pipeline vendors

Verify Akamai cache behavior with pragma headers. Confirm SFCC DIS WebP support.

3 Establish baseline CWV monitoring

Set up a Google Search Console CWV dashboard prior to Sprint 1 go-live to capture before/after measurements.

Aero

4 Implement Quick Wins (Week 1–4)

Apply fetchpriority, preload, image dimensions, and Optimizely async attribute changes. These require no platform rebuild and carry near-zero risk.

5 Audit third-party script inventory

Provide a full list of all third-party scripts currently loading site-wide, including load order and trigger conditions, to enable page-type gating implementation.

6 Validate SFCC page controller access

Confirm which SFCC controllers and ISML templates can be modified for SSR product grid, script gating, and Optimizely server-side flag delivery.

Thank you!